CS614 - Midsem Project Review

Aditi Khandelia Arush Upadhyaya Kushagra Srivastava Sankalp Mittal Vidhi Jain

Project Overview

Iterative pre-copy live migration transfers memory pages from a running process to a destination host while the process continues to execute, repeating the transfer for pages that were dirtied during the previous round until the remaining working set is small enough to complete migration with a brief final pause. The efficiency of this scheme depends entirely on the ability to identify, at the end of each round, *exactly* which pages were written. For CPU memory, the Linux kernel provides this capability through soft-dirty bits embedded in page-table entries, readable via `/proc/PID/pagemap`. No analogous mechanism exists for GPU-managed memory. NVIDIA's Unified Virtual Memory (UVM) subsystem, which underpins `cudaMallocManaged` and handles demand-paging between host and device, exposes no interface for querying which pages a process has modified. Consequently, any migration or checkpointing system must conservatively copy the *entire* GPU memory footprint on every iteration, rendering iterative pre-copy migration of large GPU workloads impractical.

This project addresses that gap by implementing **per-process dirty page tracking** directly within the open-source NVIDIA UVM kernel module. UVM already intercepts every GPU page fault to manage demand-paging; we augment the write-fault servicing path to record each written page in an in-kernel data structure, without introducing any additional hardware overhead. A `procfs`-based interface allows a privileged migration daemon to start and stop tracking epochs and to retrieve the complete set of pages written by a given process, with **page-level granularity** and a per-page **nanosecond-resolution timestamp** suitable for ordering events across migration rounds.

Group Details

Following are the details of the group members:

Roll No.	**Roll No.**	**Email ID**
Aditi Khandelia	220061	aditikh22@iitk.ac.in
Arush Upadhyaya	220213	arushu22@iitk.ac.in
Kushagra Srivastava	220573	skushagra22@iitk.ac.in
Sankalp Mittal	220963	sankalpm22@iitk.ac.in
Vidhi Jain	221191	vidhijain22@iitk.ac.in

Milestones

Milestone 1 - Midsem Review ***(4 weeks)***

1. **Literature Review, Technical Familiarization and other Preliminaries** *(1 week)*
   (a) Review existing technical documentation and architectural specifications regarding UVM.
   (b) Conduct a code review of the UVM driver and perform preliminary API experimentation.
2. **Implementation of Dirty Access Tracking in UVM Driver** *(2 weeks)*
   (a) Add instrumentation and logging to observe runtime behavior, including tracking page faults, permission transitions, migration events, and residency changes for debugging and analysis.
   (b) Implement the core dirty access tracking mechanism, including data structures for tracking dirty pages, and the logic to update these structures on write faults.

38th Conference on Neural Information Processing Systems (NeurIPS 2024).

(c) Develop a `procfs`-based interface to allow user-space tools to query the set of dirty pages for a given process, including timestamps for each page.

3. **Initial Performance Evaluation** *(1 week)*
   (a) Evaluate the performance of the modified driver on small workloads, measuring metrics such as page fault latency, migration throughput, and overall application runtime.
   (b) Identify and address any significant performance bottlenecks or correctness issues in the implementation.

Milestone 2 - Extensions, Testing and Optimisations ***(4 weeks)***

1. **Extension of Functionality** *(2 week)*
   (a) Extend the tracking mechanism to support additional features such as multiple concurrent processes, concurrent tracking on CPU and GPU side, and range-based dirty tracking to allow selective monitoring of specific virtual memory regions.
   (b) Extend the `procfs` interface to expose richer statistics, including per-page access frequency counters and dirty page age, to enable more informed migration and eviction decisions by user-space tools.
2. **Testing and Analysis of Tracking Mechanism** *(1 week)*
   (a) Test critical issues pertaining to concurrency and performance that arise under multi-process workloads, including correctness of dirty page tracking under concurrent write faults and race conditions in shared data structures.
   (b) Quantify overhead introduced by dirty tracking, measuring additional latency per write fault due to metadata updates, and memory footprint of tracking data structures such as counters, hash tables, and bitmaps.
   (c) Profile driver performance across diverse workloads with and without dirty tracking enabled, for metrics such as page fault latency, migration throughput, and overall application runtime.
3. **Optimisation of Tracking Mechanism** *(1 week)*
   (a) Based on the analysis, identify and implement optimizations to reduce overhead, such as batching updates to tracking data structures, using more efficient data structures (e.g., bitmaps instead of hash tables), and optimizing critical paths in the tracking logic.

Milestone 3 - Final Deliverables and Integration ***(2 weeks)***

1. Prepare comprehensive documentation for the tool, including usage guidelines and performance considerations.
2. Produce formal benchmarking results quantifying the tracker's performance impact across various scenarios.
3. Implement user-facing configurability for the tracker via a command-line interface (CLI).

Source Code

Development Environment

All development and testing is carried out on a dedicated virtual machine named `dirtyuvm`, running a custom kernel with the modified NVIDIA UVM driver loaded as a kernel module.

Repository

The full source code is hosted at:

`https://github.com/aleph-5/open-gpu-kernel-modules`

This is a fork of NVIDIA's open-source GPU kernel modules, with our modifications applied on top.

Modified Driver Files

All driver modifications reside under `kernel-open/nvidia-uvm/`. The files containing functionality added or significantly modified by us are:

1. `uvm_common.h` / `uvm_common.c` - The primary location of our implementation. Defines `struct dirty_page_info` (page number, timestamp, PID) and `struct uvm_dirty_page_table` (an `xarray`-based per-process table). Implements the core API: table initialisation and teardown, fault recording, and page lookup. Also declares `uvm_dirty_procfs_init` for the `procfs` interface.
2. `uvm_va_block.c` - Hooked to invoke the dirty tracking record function when a write fault is serviced on a VA block.
3. `uvm_gpu_replayable_faults.c` - Modified to record dirty page events when GPU write faults are replayed.
4. `uvm_va_space.c` / `uvm_va_space.h` - Extended to register the dirty-page GPU-unmap invalidation callback (`uvm_va_space_dirty_init`) and manage per-process tracking table lifetime alongside VA space creation and destruction.
5. `uvm.c` - Top-level integration point; initialises the procfs dirty tracking interface at module load.

Tests

User-space tests are located in the `tests/` directory at the repository root, organised into the following subdirectories:

1. `address_range_filtering` - Tests for range-based selective dirty tracking.
2. `ordering` - Tests verifying the ordering guarantees of fault recording.
3. `table_lifecycle` - Tests for correct initialisation and teardown of tracking tables across process lifecycles.
4. `latency` - Benchmarks measuring per-fault overhead introduced by the tracking mechanism.

Current Progress

Literature Review, Technical Familiarization and other Preliminaries

Implementation of Dirty Access Tracking in UVM Driver

Workflow. A tracking epoch begins when a privileged user-space tool writes to `dirty_pids_start_track`. This initialises a fresh per-process table and immediately invalidates all current GPU page-table entries by unmapping them, so that every subsequent GPU access generates a fresh page fault. Each write fault is intercepted in the driver's fault-servicing path, which records the faulting page in the table along with a nanosecond-resolution timestamp before re-establishing the mapping and replaying the faulting instruction. At any point during the epoch the tool can read `dirty_pages` to obtain the accumulated set of written pages; reads are non-destructive, so the table is not consumed by the query. The epoch ends when the tool writes to `dirty_pids_stop_track`, which frees all recorded state. Re-starting tracking destroys any existing table before creating a fresh one, cleanly beginning a new epoch.

Data Structures. The tracking state is organised as a two-level structure defined in `uvm_common.h`.

1. `struct dirty_page_info` - a per-page record holding the page number, the `ktime_get_ns()` timestamp of the first observed write fault, and the PID of the faulting process.
2. `struct uvm_dirty_page_table` - a per-process container holding an `xarray` that maps page numbers to `dirty_page_info` pointers.
3. A global `xarray` (`pid_to_page_table`) maps each tracked process's `tgid` to its `uvm_dirty_page_table`.

This two-level design provides natural per-process isolation. Entries are recorded under a *first-write-wins* policy: an existing entry is never overwritten, so the stored timestamp always reflects the earliest observed write to that page within the current epoch. The faults may be serviced by different CPU-side driver threads, thus the timestamp may not be strictly monotonic, depending upon the driver-side service thread recording the same.

APIs Exposed. User-space interacts with the tracker exclusively through five `procfs` entries created under `/proc/driver/nvidia-uvm/`:

1. `dirty_pids_start_track` (write) - initialises (or re-initialises) the tracking table for `current->tgid` and triggers GPU-mapping invalidation to start a new epoch.
2. `dirty_pids_stop_track` (write) - destroys the table for `current->tgid`, freeing all recorded entries.
3. `dirty_pid_to_query` (write) - sets the PID whose table is consulted on subsequent `dirty_pages` reads, decoupling the querying process from the tracked process.

4. `dirty_pages` (read) - iterates the selected process's table and emits one line per recorded page in the format `0x`*`addr timestamp_ns pid`*. Reads are *non-destructive*: entries remain in the table and can be queried repeatedly within the same epoch.
5. `dirty_range` (write) - accepts a pair of hexadecimal addresses `0x`*`start`* `0x`*`end`* (end exclusive) to restrict subsequent `dirty_pages` reads to a specific virtual address window. The range is protected by a spinlock to allow concurrent updates safely. An inverted or zero-length range produces an empty result rather than undefined behaviour.

Internal Functions and Implementation Sites. The implementation is spread across several driver files, each responsible for a distinct concern:

1. `uvm_common.c` - the primary implementation file. Contains `uvm_dirty_page_table_init` / `uvm_dirty_page_table_destroy` (table lifecycle), `uvm_dirty_page_table_record` (fault recording, using `GFP_ATOMIC` as it executes in a non-sleepable context), `uvm_dirty_page_table_lookup`, and the full `procfs` handler implementations including `uvm_dirty_procfs_init`.
2. `uvm_gpu_replayable_faults.c` - the write-fault hook. After standard fault servicing, checks `uvm_dirty_tracking_active_for_pid(va_block->creator_pid)` and calls `uvm_dirty_page_table_record` for `UVM_FAULT_ACCESS_TYPE_WRITE` faults.
3. `uvm_va_space.c` - implements `uvm_dirty_invalidate_all_gpu_mappings`, which walks all live VA spaces and blocks to unmap GPU PTEs at epoch start. Exposes `uvm_va_space_dirty_init` to register this function via the `uvm_dirty_invalidate_fn` function pointer at module load.
4. `uvm.c` - top-level integration; calls `uvm_dirty_procfs_init` and `uvm_va_space_dirty_init` during module initialisation.

Initial Performance Evaluation

Next Steps and Planned Progress

Multi-process isolation and range-based dirty tracking (the `dirty_range` procfs interface) are fully implemented. The test suite covers table lifecycle, ordering guarantees, address range filtering, and fault-to-visibility latency. The remaining work is as follows.

Extensions Still Pending

1. **CPU-side Dirty Tracking.** The current implementation intercepts GPU write faults only. Extending tracking to CPU-side writes requires integration with the `mmu_notifier` invalidation path so that CPU dirty pages are recorded in the same per-process table and exposed through the existing procfs interface.
2. **Per-page Access Frequency Counters.** The tracker currently operates under a first-write-wins policy: each page is recorded once with its earliest observed write timestamp. Richer statistics, such as per-page write counts and dirty page age, are needed to support informed migration and eviction decisions by user-space tools. This requires extending `struct dirty_page_info` with a counter field and updating the fault hook to increment it atomically.

Testing and Performance Analysis

1. **Overhead Quantification.** The per-fault overhead introduced by the tracking path (xarray lookup and conditional insert under `GFP_ATOMIC`) has not been formally benchmarked. Write fault latency will be measured with and without tracking enabled across a range of workload sizes to bound the cost.
2. **Memory Footprint Profiling.** Under workloads with large dirty sets, the two-level xarray structure may incur non-trivial memory overhead. The footprint of the tracking data structures will be measured and compared against alternatives such as bitmaps.
3. **Concurrency Stress Testing.** The existing concurrent-stream test (`tc06_concurrent_streams`) exercises the xarray under parallel GPU faults. Additional stress tests under multi-process workloads and simultaneous range-filter updates are needed to validate the spinlock boundary.

Optimisation

Based on profiling results, targeted optimisations will be applied. Likely candidates include batching xarray inserts to reduce per-fault locking overhead, and replacing the xarray with a bitmap representation for workloads where the dirty set density is high enough to justify the fixed memory cost.

Further Deliverables

1. **Documentation.** Comprehensive usage documentation covering the procfs interface, epoch semantics, range filtering, and known limitations.
2. **Formal Benchmarking Report.** Quantified results for fault latency overhead, memory footprint, and migration throughput impact across representative workloads, with and without the tracker active.
3. **Command-line Interface.** A user-facing CLI tool that wraps the procfs interface, providing commands to start and stop tracking, query dirty pages for a given PID, set address range filters, and display results in a human-readable format.

Declaration

We declare that the work presented in this report is our own, except where otherwise acknowledged. We have also read and understood the University's policy on plagiarism and academic integrity, and we confirm that this report complies with those standards.